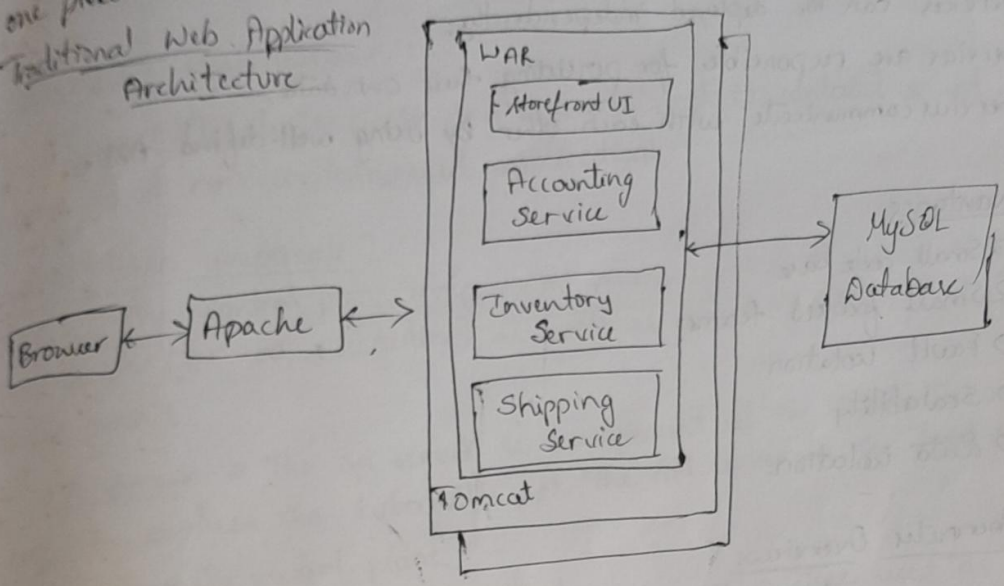
Monolithic Architecture

It is the traditional unified model for the design of a software program. Monolithic, in this context, means "composed all in one piece".

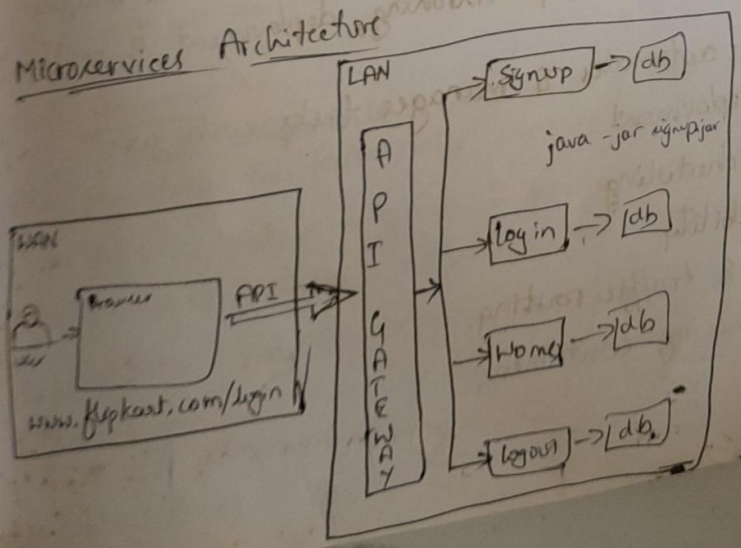
Traditional Web Application Architecture



Drawbacks

- → Tight coupling between components
- → Less reusability
- → Large code base
- → Less scalable
- → More deployment time & restart time

Microservices Architecture



Micro services - also known as the microservice architecture - is an architectural style that structures an application as a collection of services

- Microservice are small, independent, and loosely coupled.
- Each service is a separate codebase.
- Services can be deployed independently.
- Services are responsible for persisting their own data.
- Services communicate with each other by using well-defined APIs.

Advantages

- Small code base
- Small focused teams
- Fault isolation
- Scalability
- Data isolation.

Kubernetes Overview:

It is an open source container archestration tool which automates container deployment, container scaling & load balancing.

It schedules, runs and manages isolated containers which are running on virtual/physical cloud.

All top cloud providers support Kubernetes.

Container orchestration:

It is a process of manage and automate the complete lifecycle of containers, including provisioning, deployment & scaling.

Container orchestration: automates & manages tasks such as:

provisioning & deployment
Configuration & scheduling
Container availability
Load balancing & traffic routing.
Scaling & removing containers

Features of Kubernetes:

load balancing.
orchestration
Auto scaling
Platform independent
Fault tolerence (node/pod)
Automated rollbacks
Health monitoring
secret & configuration management.

Kubernetes: Architecture:

Kubernetes cluster: It is a network and it contains a set of nodes that run containerized applications.

Control Plane components:

The control plane components make global decisions about the cluster (for ex, scheduling), as well as detecting & responding to cluster events.

API server → The api server is a component of the Kubernetes control plane the exposes the kubernetes api. The API server is the front end for the kubernetes control. plane.

Etcd:- consistent and highly-available key value store used as kubernetes backing store for all cluster data.

Kube-scheduler → It is control plane process which assigns Pode to Nodes.

Kube-controller-manager :→ Control plane ~~process which assigns Pods to nodes~~. component that runs controller processes. Logically, each controller is a separate process, but to reduce complexity, they are all compiled into single binary & run in a single process.

Node Components:

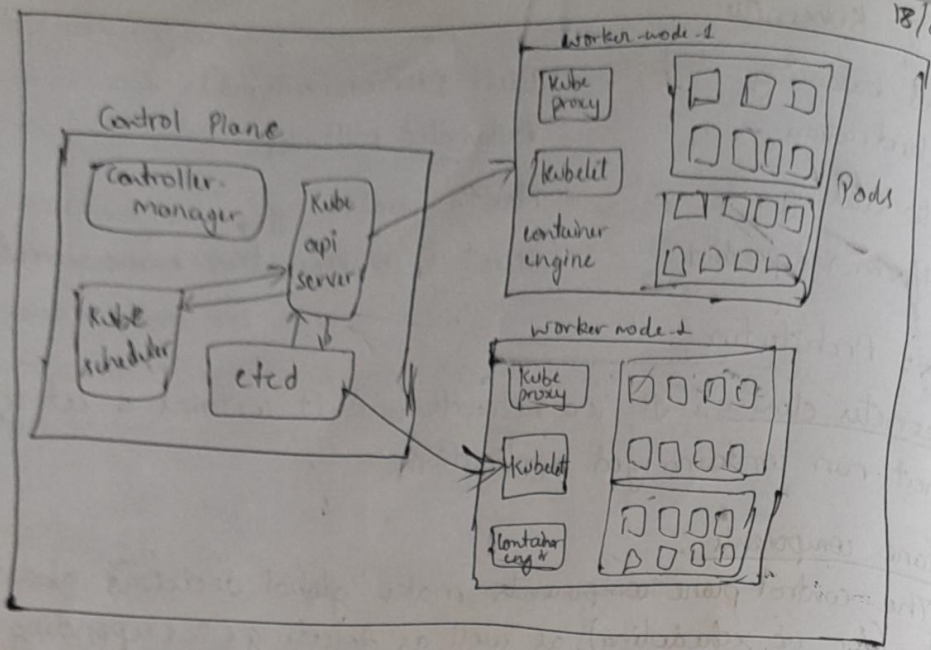
Node components run on every node, maintaining running pod and providing the kubernetes runtime environment.

Pod: a pod is a group of one or more containers, with shared storage and network resources, and a specification.

Kubelet: The kubelet is the primary "node agent" that runs on each node. It can register the node with the api server using one of the hostname i.e IP address.

Kube-proxy: Kube-proxy is a network proxy that runs on each node in your cluster.

18/09/25



create 1 pod with one container with pod name devops & image name ubuntu & print the content Hello

.yml file

```
apiVersion: v1
kind: Pod
metadata:
    name: devops
spec:
  containers:
    - name: signup
      image: ubuntu
      command: ["/bin/bash", "-c", "while true; do echo hello;
                sleep 2; done"]
```

To run that file

→ kubectl apply -f pod.yml (filename)

To check pods.

→ kubectl get pods.

To get pods ip

→ kubectl get pods -o wide

To describe pod object

→ kubectl describe pod devops

to troubleshoot go for events by describing them

to go into pod

→ kubectl exec -it devops -- /bin/bash

→ exit

to check with os/image running in container

→ kubectl exec -it devops -- cat /etc/os-release

to delete all resources of pods

→ kubectl delete -f pod.yml

to check container logs

→ kubectl logs -f devops

---

create 1 pod with 2 containers pod name = flipkart & container 1 signup container 2 login with image ubuntu for both. & signup will print signup login will print login

```
apiVersion : v1
kind : Pod
metadata:
    name : devops
spec:
    containers:
        - name: signup
          image : ubuntu
          command : "~~echo signup~~" [same as above]
    ~~containers~~:
        - name : login
          image : ubuntu
          command : "~~echo login~~" [same as above]
```

To get into particular container

→ kubectl exec -it [pod-name] -c [container name] -- /bin/bash

To get logs of particular container

→ kubectl ~~logs~~ -f devops -c [containername]

curl c]

-> Create 1 pod with 1 container with name pod webapp & assign port no to container

-> no resources found in default namespace.

```
apiVersion: v1
Kind: Pod
metadata:
    name: webapp
spec:
  containers:
    - name: login
      image: httpd
      ports:
        - containerPort: 80
```

To get list of Namespace

-> kubectl get ns (or) kubectl get namespace

To create namespace

```
apiVersion: v1
Kind: Namespace
metadata:
    name: dev
```

-> To appy this

kubectl apply -f " "

-> To run the pods in particular namespace

kubectl apply -f "pod.yml" -n "namespace"

-> To get pods in particular namespace

kubectl get pods -n "namespace"

-> To delete namespace

kubectl remove -f "ns.yml"

Label Selectors:

These are used to filter pods, resources, based on the condition. We have 2 types of label selectors.

- Equality based
- Set based

```
apiVersion: v1
kind: Pod
metadata:
  name: webapp
  labels:
    batch: archos
    env: dev

spec:
  containers:
    - name: login
      image: httpd
      ports:
        - containerPort: 80
```

To get labels information of pods

kubectl get pods --show-labels

To assign label after creating pods

Kubectl label pod (name) team=rcb

To filter pods using selectors

kubectl get pods -l team=rcb.

Communication between container to container in the same pod.

```
apiVersion: v1
kind: Pod
metadata:
  name: webapp
spec:
  containers:
    - name: signup
      image: amazonlinux
      command: ["/bin/bash", "-c", "while true; do echo signup; sleep 1;
done"]
```

```
- name: login
  image: httpd
  ports:
    - containerPort: 80
```

Replication Controller

It ensures a specified number of pods replicas are always running, helping with fault tolerance and scalability.

```
apiVersion: v1
kind: ReplicationController
metadata:
  name: flipkart
spec:
  replicas: 2
  selector:
    app: nginx
  template:
    metadata:
      name: dev
      labels:
        app: nginx
    spec:
      containers:
        - name: login
          image: nginx
          ports:
            - containerPort: 80
```

```
kubectl apply -f [ ]
kubectl get rc
kubectl get pods
kubectl describe rc [name]
kubectl delete pod [Pod name]
kubectl scale --replicas=4 rc [rc name]
kubectl scale --replicas=2 rc [rc name]
kubectl scale --replicas=4 rc -l app=nginx
kubectl scale --replicas=1 rc -l app=nginx
```

Replicaset:
It is the advanced version of replica Controller

```
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: amazon

spec:
  replicas: 3
  selector:
    matchExpressions:
      - {key: myname, operator: In, values: [signup, login, home]}
      - {key: env, operator: NotIn, values: [amazon]}

  template:
    metadata:
      name: devops
      labels:
        myname: signup

    spec:
      containers:
        - name: signup
          image: httpd
          ports:
            - containerPort: 80
```

---

kubectl apply -f [name]

kubectl get rs

kubectl get pods

kubectl describe rs [rs name]

kubectl delete pod [pod name]

kubectl scale --replicas=6 rs [ ]

✗ kubectl scale --replicas=8 rs -l ~~app=~~ myname=signup

error: no objects passed to scale

1) Difference between replication Controller & replicaset

Deployment:
deployment declaratively manages set of pods, facilitating easy and rollbacks.

```
apiVersion: apps/v1
kind: Deployment
metadata:
    name: flipkart
spec:
    replicas: 3
    selector:
      matchlabels:
        name: deployment
    template:
      metadata:
        name: signup
        labels:
          name: deployment
      spec:
        containers:
          - name: v1
            image: ubuntu
            command: ["/bin/bash", "-c", "while true; do echo hello; done"]
```

-> kubectl get deploy
-> kubectl get rs
-> kubectl get all
-> kubectl rollout status deployment flipkart -> go back to previous version
-> kubectl rollout undo deployment flipkart

Kubernetes Networking
Mapping all the pods & expose the pods from outside the cluster

Service: it is a method of exposing a network application that is running in one or more pods in the cluster
To perform kubernetes networking deployment objects is compulsory.

In the kubernetes it has 3 types of service
1) ClusterIP - it is a internal IP & a default IP
2) It is used to map multiple pods running in multiple worker nodes

2) node Port:
Clients send requests to the ipaddresses of node, one or more node port values that are specified by the service

nodePort range from 30000 to 32,767.

3) Load Balancer: It is used to distribute the load b/w multiple servers under specified service.

```
---
apiVersion: apps/v1
kind: Deployment
metadata:
    name: flipkart

spec:
    replicas: 2
    selector:
        matchLabels:
            name: deployment

    template:
        metadata:
            name: signup
            labels:
                name: deployment

        spec:
            containers:
                - name: n2
                  image: nginx
                  ports:
                    - containerPort: 8000
```

```
---
apiVersion: v1
kind: Service
metadata:
    name: flipkartsignup
spec:
    ports:
      - port: 80
        targetPort: 8000
    selector:
        name: deployment
    type: ClusterIP
```

```
kubectl apply -f deploy.yml
kubectl get deploy
kubectl get rs
kubectl get pods
kubectl get svc/service
```

nodePort

kubectl apply -f deploy.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
    name: flipkart
spec:
    replicas: 2
    selector:
        matchlabels:
            name: deployment
    template:
        metadata:
            name: signup
            labels:
                name: deployment
        spec:
            containers:
                - name: v1
                  image: nginx
                  ports:
                    - containerPort: 80
---
apiVersion: v1
kind: Service
metadata:
    name: service signup
spec:
    ports:
      - port: 80
        targetPort: 80
        nodePort: 30020
    selectors:
        name: deployment
    type: nodePort
```

Load Balancer

```
apiVersion: apps/v1
kind: Deployment
metadata:
    name: flipkart
spec:
    replicas: 2
    selector:
        matchLabels:
            ~~name: deployment~~ app: myapp
    template:
        metadata:
            name:
            labels:
                app: myapp
```

```
spec:
  containers:
    - name: signup
      image: nginx
      ports:
        - containerPort: 80

---

apiVersion: v1
kind: Service
metadata:
  name: flipkartsignup

spec:
  selector:
    app: myapp
  ports:
    - protocol: TCP
      port: 80
      targetPort: 80
      nodePort: 30070
  type: LoadBalancer
```

---

to perform load balancing

we need to create target

↳ target groups

↳ 3 ip addresses

↳ target name

[ ] any

↳ Protocal : Port

TCP [ ] 1-65535

↳ IPv4

↳ ~~add~~ Step: 2 give private ip of worker nodes

include as pending below

create target group

Create Load balancing

↳ Network load balancing

↳ Internal

↳ IPv4

↳ Select all regions

↳ Security group

↳ add new inbound rule

ie ALLTCP & anywhere IPv4

↳ Listening & routing

Protocol	Port
TCP	30070

select your target group

Create

27/03/25

Volumes

Kubernetes volumes is used to share the resources between multiple resources like containers, pods, & nodes.

Kubernetes volumes provide way to persist & share data between containers to containers within the pods, to share resources b/w pod to pod and to share the resources b/w node to node.

In kubernetes volume it has 3 types

1) emptyDir
2) hostPath
3) persistentVolume or persistentVolumeClient

1) emptyDir : it is used to share the resources between containers to container within the pod

to create volume

```
volumes:
  - name: cak
    emptyDir: {}
```

to attach volume to container

```
volumeMounts:
  - name: cak
    mountPath: "/temp/signup"
```

```
apiVersion: v1
kind: Pod
metadata:
  name: flipkart

spec:
  containers:
    - name: signup
      image: centos
      command: ["/bin/bash", "-c", "sleep 15000"]
      volumeMounts:
        - name: sak
          mountPath: "tmp/signup"
    - name: login
      image: centos
      command: ["/bin/bash", "-c", "sleep 15000"]
      volumeMounts:
        - name: sak
          mountPath: "tmp/login
  volumes:
    - name: sak
      emptyDir: {}
```

```
kubectl apply -f [ name ]
kubectl get pods
kubectl exec -it flipkart -c signup -- /bin/bash
  > cd /tmp/signup
  > ls
  > echo "logs of signup" >> signup.txt
  > ls
  > exit
kubectl exec -it flipkart -c login -- /bin/bash
  > cd /tmp/login
  > ls
  > cat signup.txt
  > exit
```

---

hostPath: ~~It is used to share~~ host path volumes mounts the from the host node file system into a pod providing access to host level files or directories

=> it is used to share the resources between pod to pod in the same node

-> where pod is running in that node only hostpath volume will be created.

create hostpath volume

```
volumes:
  - name: cak
    hostPath:
      path: /tmp/data


apiVersion: v1
kind: Pod
metadata:
  name: flipkart
spec:
  containers:
    - name: cigroup
      image: centos
      command: "/bin/bash", "-c", "sleep 18000"]
      volumeMounts:
        - name: cak
          mountPath: /tmp/hostpath
  volumes:
    - name: cak
      hostPath:
        path: /tmp/data.
```

ConfigMap & Secret

configmap is an api object used to store non confidential data in key value pairs

Pods can consume configmaps as a environment variables, command line arguments or as configurations files in a ~~module~~ volume

Creation of configmap [cm]

name of cm

kubectl create configmap [mymap] --from=sample.conf

kubectl get configmap

kubectl delete ~~my~~configMap mymap

```
apiVersion: v1
kind: Pod
metadata:
  name: myconfig
spec:
  containers:
  - name: c1
    image: centos
    command: ["/bin/bash", "-c", "while true; do echo welcome to k8s; sleep 2;
                done"]
    volumeMounts:
      - name: testconfigmap
        mountPath: "/tmp/config"
volumes:
- name: testconfigmap
  configMap:
    name: mymap
    items:
      - key: sample.conf
        path: sample.conf
```

Secrets:

"The secret is an object to store sensitive information or confidential data like credentials and it can be mounted into pods.

Create secret object

```
kubectl create secret generic mysecret --from-file=user.txt --from-file=pass.txt
kubectl get secret
kubectl describe secret mysecret
kubectl delete secret mysecret
```

```
apiVersion: v1
kind: Pod
metadata:
  name: mysecret
spec:
  containers:
  - name: c1
    image: centos
    command: ["/bin/bash", "-c", "while true; do echo welcome to devops; sleep 5000; done"]
    volumeMounts:
      - name: testsecret
        mountPath: "/tmp/mysecrets"
volumes:
  - name: testsecret
    secret:
      secretName: mysecret
```

Pods Resources management

```
apiVersion: v1
kind: Pod
metadata:
  name: resources
spec:
  containers:
  - name: resources
    image: centos
    command: ["/bin/bash", "-c", "while true; do echo welcome to devops; sleep 5; done"]
    resources:
      requests:
        memory: "64Mi"
        cpu: "100m"
```

limits:
memory: "128Mi"
cpu: "900m"

04/03/25